ISSN 1064-2269, Journal of Communications Technology and Electronics, 2021, Vol. 66, No. 12, pp. 1336–1345. © Pleiades Publishing, Inc., 2021.
Russian Text © The Author(s), 2021, published in Radiotekhnika i Elektronika, 2021, Vol. 66, No. 12, pp. 1178–1188.

THEORY AND METHODS OF SIGNAL PROCESSING

ASMD–FSMD Technique in Designing Signal Processing Devices on Field Programmable Gate Arrays

V. V. Solov'ev*

Belarusian State Academy of Communications, Minsk, 220076 Belarus

**e-mail: valsol@mail.ru*

Received March 20, 2021; revised April 7, 2021; accepted April 10, 2021

Abstract—The algorithmic state machine with a datapath (ASMD)–finite state machine with a datapath (FSMD) technique for designing digital devices is discussed, in which the operation of a device is described in the form of a block diagram of a state machine with a datapath. Different techniques for designing digital devices are compared by the examples of synchronous multipliers and peripheral interface controller (PIC) processors on field programmable gate arrays (FPGAs). It is shown that the ASMD–FSMD technique, in contrast to the conventional approach, in most cases makes it possible to reduce the sales value (by 47% for particular examples) and significantly enhance the speed (by a factor of 2.96 for particular examples), as well as to essentially shorten the design time (by a factor of 5–7). Recommendations on using the ASMD–FSMD technique are given and possible directions for its further development are pointed.

DOI: 10.1134/S1064226921120184

INTRODUCTION

Recently, digital signal processing systems have become increasingly complex at the simultaneous strengthening of requirements for the design time and reliability of projects. A way of solving this problem is the application of new techniques for designing digital devices to engineering practice.

A designed digital device is conventionally presented in the form of a datapath and a control unit, which are usually designed separately: the former, as a set of standard functional units (registers, buses, multiplexers, etc.) and the latter, as a finite state machine (FSM).

Gajski et al. [1], proposed to combine the datapath and control unit and represent them as an FSM with a datapath (FSMD). The FSMD model has quickly become popular; FSMD options for synchronous and asynchronous designs were presented in [2]. The FSMD model turned out to be very convenient for checking the equivalence of two circuits obtained by synthesis or various design transformations [3, 4]. Schaumont et al. [5], proposed to present a digital system in the form of an FSMD network, which makes it possible to implement race-free equipment.

The general FSMD model is not always convenient for designing specific products. Therefore, in some studies, FSMD extensions were proposed: for presenting the processor architecture and application-specific integrated circuit (ASIC) [6], for synchronous memory access [7], and for array processing software [8]. The reduction of power consumption by decomposing the FSMD [9] and by gating [10] was considered. FSM and FSMD efficiencies were compared in [11].

Owing to their clarity, algorithmic state machines (ASMs) have become widespread in representing the operation algorithms of FSMs. ASMs were first proposed in [12] as an alternative to automaton graphs. Green et al. [13], considered ASM implementation on PROM, FPLA, and multiplexers and Baranov [14] proposed methods for minimizing the number of ASM vertices. Jenihhin et al. [15], describes the ABELITE tool for synthesizing ASM-based controllers.

ASMs are conventionally used to represent the operation algorithm for a control device. Ciletti [16] proposed using ASMs for describing both the behavior of a control device and describing the register operations performed in an operational device. Such an ASM is called an algorithmic state machine with a datapath (ASMD). ASMD diagrams have recently been increasingly used in FPGA designs in the implementation of industrial control systems [17], implementation of the asin function using the CORDIC algorithm [18], hardware implementation of the advanced encryption standard (AES) [19], and designing a universal asynchronous UART transceiver [20].

In this study, the ASMD–FSMD technique based on the Verilog language is investigated in designing synchronous multipliers and PIC processors on FPGAs. In the ASMD–FSMD technique, the opera-

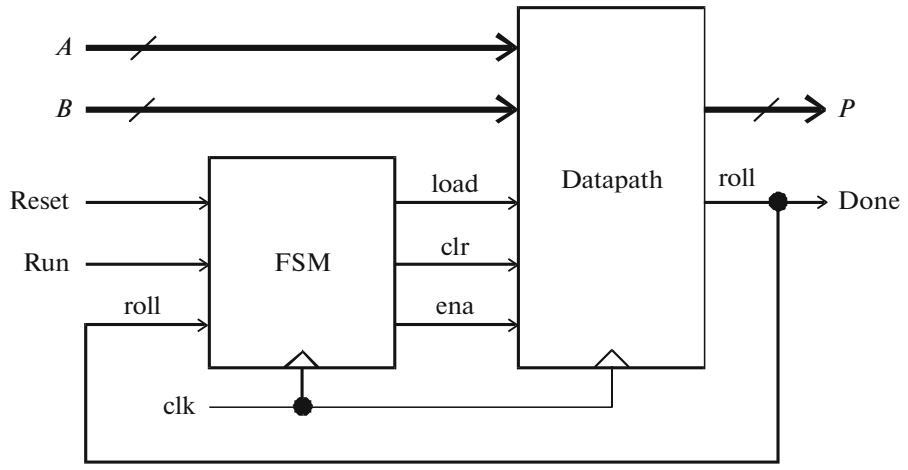


Fig. 1. Block diagram of a synchronous multiplier in the form of a datapath operational and a control unit.

tion of a device is presented in the form of an ASMD scheme, which is described in the Verilog language as an FSMD.

1. CONVENTIONAL APPROACH TO THE DESIGN OF DIGITAL DEVICES

We consider the simplest school multiplication algorithm that performs arithmetic operation $P = A \times B$ of multiplication of two binary numerals with a width of N bits. First, P is zeroed. In each multiplication cycle, the value of the least significant bit of the multiplier $B[0]$ is checked. If $B[0] = 1$, then product P is added with factor A. If $B[0] = 0$, then product P is added with zero or with nothing. After that, the content of multiplier B is shifted one bit to the right and the value of product P, to the left. The algorithm ends after considering all bits of multiplier B.

Figure 1 shows a block diagram of a synchronous multiplier in the form of a datapath and a control device (FSM). The A and B values are supplied to the datapath input. At the datapath outputs, product P and signal *done* indicating the end of multiplication are formed. In addition, the datapath generates internal signal *roll* coinciding with signal *done*. The FSM control unit generates the following control signals: signal *load* for loading the values of multiplied words A and B into datapath registers, signal *clr* for resetting the datapath registers, and signal *ena* for enabling the operation of shifting the contents of the shift registers.

Figure 2 shows a diagram of the datapath that implements the multiplication algorithm. The datapath includes left shift register *ra*; right shift register *rb*; register *rp* for storing words A, B, and P, respectively; a 2-1 bus multiplexer; a $2N$-bit adder; and a modulo counter, which generates signal *roll*.

The operation of the synchronous multiplier control unit can be presented as a Mealy or a Moore machine (Figs. 3a and 3b, respectively). Note that a Mealy FSM contains two states, S_0 and S_1, while a Moore FSM, contains three states, S_0, S_1, and S_2. The details of describing the datapath components (Fig. 2) and the Mealy and Moore FSMs (Fig. 3) in the Verilog language in the conventional implementation of a synchronous multiplier were presented in [21].

2. ASM BLOCK DIAGRAMS

The ASM block diagram is built to visualize the description of the FSM operation algorithm and represents a directed connected graph [12] containing vertices of three types:

(i) Rectangles as tops of states (state box).

(ii) Rhombs as conditional vertices (decision box).

(iii) Ovals as conditional output vertices (conditional output box).

The top of the ASM state (rectangle) defines the state of the machine. Near the top of the state, the name of the state (for example, S0, START, INITIAL, etc.) and its binary code can be written. In the Moore state machine the output signals that take a unit value in this state are written inside the state vertex. By default, all the remaining output signals are assumed to be zero in this state.

The conditions to be checked are written in conditional ASM vertices (rhombs); the conditional ASM vertex is the algorithm branch point. The outputs of the conditional vertex are denoted by 0 and 1 corresponding to transitions in the case of a 0 (false) or 1 (true) value of the condition check result. The condition can be an FSM input variable, a logical expression, a single bit of a bit vector, etc.

At the vertices of the conditional outputs (ovals), the output signals of the Mealy state machine are written, which take a unit value at a certain transition. In the ASM, there are no conditional outputs vertices (ovals) for the Moore state machines and, in the ASM,

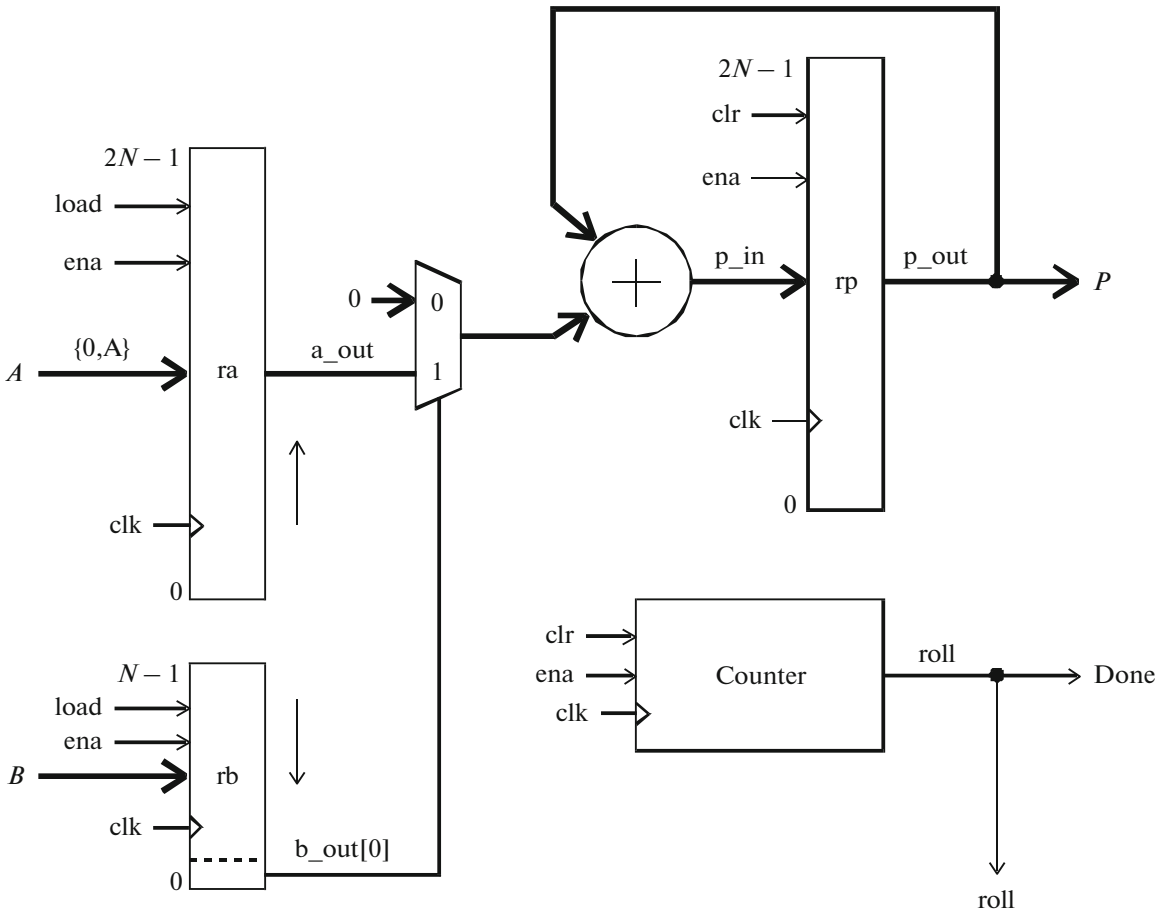


Fig. 2. Scheme of the synchronous multiplier datapath.

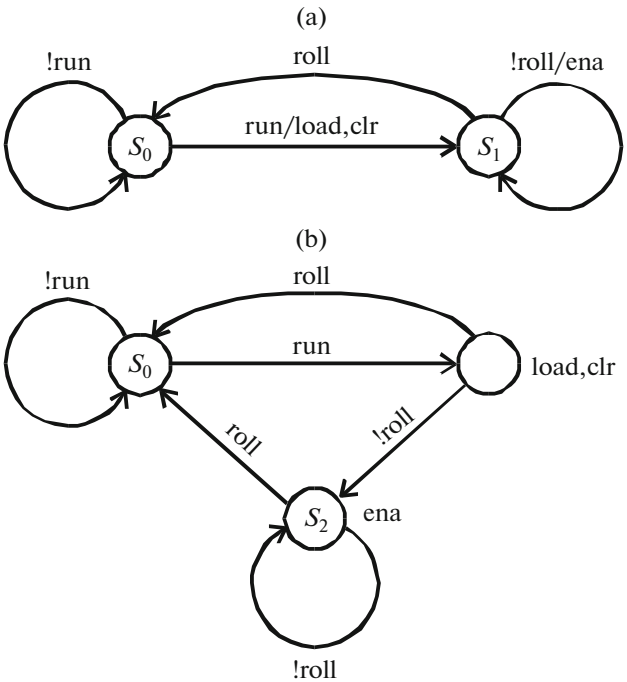


Fig. 3. Representation of a synchronous multiplier control unit in the form of a finite (a) Mealy and (b) Moore machine graph.

nothing is written at the vertices of states for the Mealy state machines.

The main building block of the ASM diagram is the ASM block (see the example in Fig. 4). The ASM block describes the behavior of the machine in one state during one synchronization cycle. Therefore, the ASM block includes only one vertex of the state (rectangle) and can have several conditional vertices (rhombs) and conditional output vertices (ovals); the rhombs can either precede the ovals or follow them. The inputs and outputs of the vertices are connected by arcs. The ASM block has only one input, which is the input to the top of the state, and can have one or more outputs. Feedbacks inside the ASM block are forbidden. The algorithm loops and pending vertices in the ASM are implemented using external feedbacks.

The ASM block diagram is a composition of interconnected ASM blocks. Each output of any ASM vertex can be connected to only one input of another vertex; i.e. the algorithm can only branch at the conditional vertices.

Note some natural features of the ASM. It preserves the clarity of the algorithm diagrams. In the ASM, the internal states of the machine are explicitly

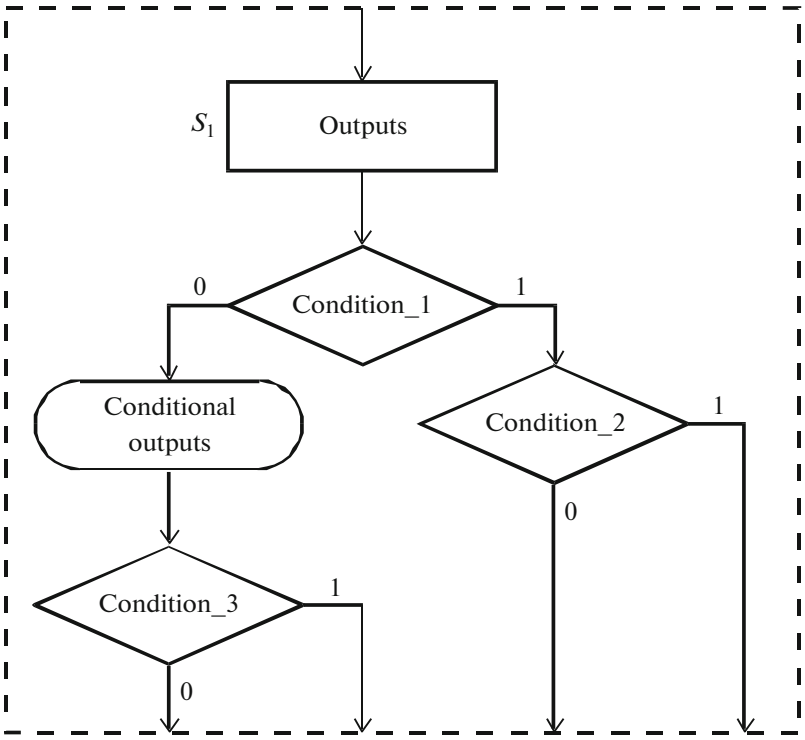


Fig. 4. ASM unit.

determined using ASM blocks. The transition from one ASM block to another is always performed in one clock cycle; i.e., the operation algorithm of the machine is directly connected to the automaton time. Therefore, by the ASM, one can always trace how many clock cycles are needed to perform an algorithm fragment.

3. ASMD–FSMD TECHNIQUE

The ASMD block diagram is an ASM in which one can write any operation on registers that are allowed in the Verilog language in rectangles and ovals and any logical expressions of the Verilog language can be checked at the conditional vertices. The ASMD diagram, similar to the ASM one, consists of blocks. The actions described within the ASMD block are performed within one clock cycle. The ASMD state machine is called an FSM with datapath (FSMD). We present the ASMD–FSMD technique in the form of the following algorithm.

Algorithm. ASMD–FSMD technique for designing digital devices.

Step 1. The FSMD states are determined.

Step 2. An ASMD unit is built for each state.

Step 2.1. Logical functions are written in the conditional vertices of the ASMD, the values of which are checked in this state.

Step 2.2. For the Moore FSMD, the operations performed with the register contents in this state are recorded in its vertices (rectangles).

Step 2.3. For the Mealy FSMD, the operations performed with the register contents at the transition are recorded at the vertices of the conditional outputs (ovals).

Step 3. The ASMD blocks are interconnected according to the device operation algorithm. In this case, each output of the ASMD unit can be connected to only one input of this or another ASMD unit.

Step 4. If necessary, the ASMD optimization is performed.

Step 5. The FSMD code is built in the Verilog language directly from the ASMD. In the code, the ASMD variables correspond to registers or triggers (for one-bit variables). The logical functions tested at the conditional vertices of the ASMD correspond to the logical expressions in the *if* statements. The actions performed in the ASMD units are described as procedure blocks *begin...end*. The operations performed in the ASMD rectangles (for the Moore machines) are described at the beginning of the block *begin...end* and the operations performed in rhombs (for the Mealy machines), in the appropriate places of the *if* statements (possibly, using the *begin...end* operator brackets).

Step 6. The FSMD is implemented using an appropriate design tool.

Step 7. End.

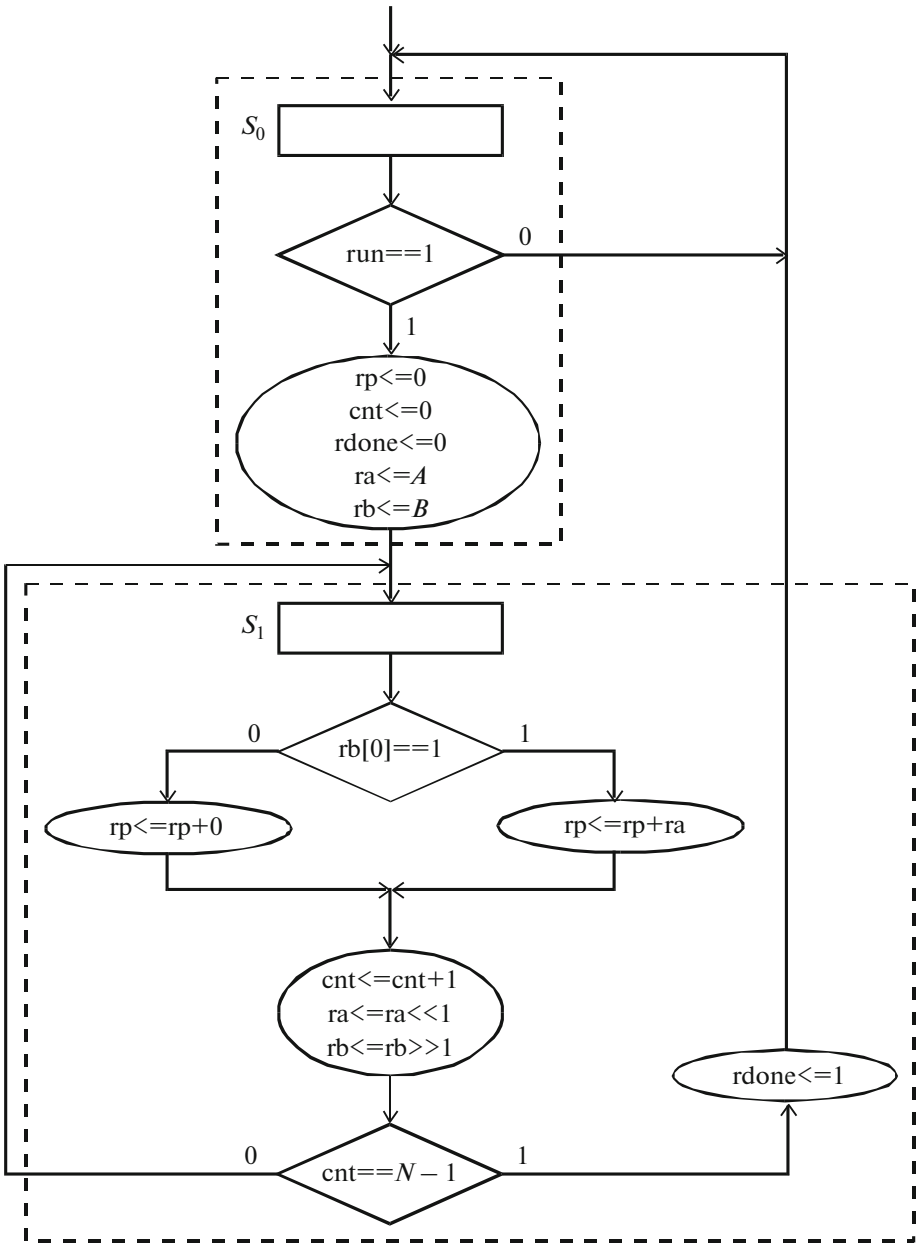


Fig. 5. ASMD circuit for the Mealy machine that ensures the higest operation speed of a synchronous multiplier.

The main stage of the ASMD–FSMD technique is building the ASMD on the basis of the device operation algorithm. The representation of the operation of a digital device in the form of an ASMD resembles more the algorithmic description of the device [21]. However, in contrast to the algorithmic description, the ASMD explicitly determines the FSMD states that can correspond to the states of the control device. In contrast to the conventional approach, the ASMD is free of a strict separation between the datapath and control unit; in addition, the structure of the datapath is not explicitly determined in the ASMD. Moreover, the presented ASMD–FSMD technique makes it possible to implement both the Mealy and Moore machines, as well as their combined model (in [16], the ASMD was only considered for the Moore machine).

In addition, it should be noted that the same device operation algorithm can be described by different ASMDs, which affects the operation speed and implementation cost of the design. To increase the operation speed, the algorithm loops should be described with the minimum number of states in order to minimize the number of states in the loop path. The Mealy machines are better suited for this purpose.

Figure 5 shows the ASMD corresponding to the Mealy FSMD for implementing the investigated synchronous multiplier. A feature of the ASMD diagram

in Fig. 5 that it contains only two ASMD blocks: in state S_0, the registers are initialized and state S_1 corresponds to one multiplication cycle.

It can be seen in Fig. 5 that one multiplication cycle is performed in one clock cycle; therefore, multiplication of N-bit binary numbers by the ASMD is performed in n clock cycles ($n = N + 1$).

We note the differences of the ASMD–FSMD technique from the available approaches to the design of digital devices. The ASMD–FSMD technique allows one:

(i) To describe both Mealy and Moore machines (in [16], ASMDs only describe the Moore machines), as well as the combined Mealy–Moore machine models.

(ii) To combine the description of the operational and control devices in one ASMD diagram (in [16], the ASMDs describe the datapaths and control units separately).

(iii) To describe a digital device in the Verilog language using one module, which significantly decreases the design time and improves the project reliability.

(iv) To involve programmers and mathematicians in the development of digital devices who are not familiar with the intricacies of designing electronic devices.

4. EXPERIMENTS

The efficiency of the ASMD–FSMD technique was investigated with the implementation of the well-known multiplication methods on the Cyclone IV E FPGA using the Quartus Prime 18.1 software [22]. The examined classical multiplication methods were:

(i) *Algorithm a,* in which the least significant bit of the multiplier is checked, the multiplier is shifted to the right, and the multiplicand is shifted to the left (the above-mentioned school algorithm).

(ii) *Algorithm b*, in which the most significant bit of the multiplier is checked, the multiplier is shifted to the left, and the multiplicand is shifted to the right.

(iii) *Algorithm c*, in which the most significant bit of the multiplier is checked and the multiplier and partial products are shifted to the left.

(iv) *Algorithm d*, in which the least significant bit of the multiplier is checked and the multiplier and partial products are shifted to the right.

In addition, the Booth multiplication algorithm and the modified Booth multiplication algorithm (six multiplication methods in total) were investigated.

Each multiplication method was implemented using the conventional approach in the form of a datapath and a control unit and the ASMD–FSMD technique (12 projects in total). The projects were investigated with input word widths of 4, 8, 16, 32, 64, and 128 bits (72 examples).

The experimental results are given in Table 1.

Analysis of Table 1 shows that, for most examples, the ASMD–FSMD technique can reduce the implementation cost and increase the operation speed. At the same time, in some cases, the implementation cost is reduced by a factor of 1.47, i.e., by 47% (the example is *mult_a_128*) and the operation speed increases by a factor of 2.96 (the example is *mult_d_16*). These values are given in bold.

To investigate the ASMD–FSMD technique in more complex designs, the following PIC processor designs were created [21]:

(i) Single-cycle processor PIC *PIC_1_cycle_N*.

(ii) Multi-cycle PIC processor *PIC_multi_N*.

(iii) Two-cycle PIC processor *PIC_2_cycle_N*.

(iv) Four-cycle processor PIC *PIC_4_cycle_N,*

where N is the data bus width (bits).

Each design was implemented using the conventional approach and the ASMD–FSMD technique (8 designs in total). The PIC processor designs were tested at data bus widths N of 4, 8, 16, 32, 64, and 128 bits (48 examples in total). The results of the investigations of the PIC processor designs are given in Table 2.

Analysis of Table 2 shows that, in designing the PIC processors, the ASMD–FSMD method is inferior to the conventional approach in the implementation cost, but, in most cases, wins in operation speed: by a factor of 1.46 for some examples (design *PIC_2_cycle_128*). This value is given in bold.

The arithmetic means of the ratios L_T/L_A and t_T/t_A for each multiplication method are given in Table 3.

It follows from Table 3 that the ASMD–FSMD technique allows, on average, to reduce the implementation cost for most multiplication methods, except for methods *mult_c* and *mbooth*, and increase the operation speed for all the methods. An increase in the operation speed is especially noticeable for methods *mult_c* and *mult_d* (given in bold).

The analogous average ratios L_T/L_A and F_T/F_A for the PIC processor designs are given in Table 4. Analysis of Table 4 shows that the use of the ASMD–FSMD technique makes it possible to increase the operation speed of PIC processors by a factor of 1.1 (by 10%) on average and, for two-cycle PIC processors, by a factor of 1.26 (by 26%) (given in bold). The ASMD–FSMD technique is inferior to the conventional approach in the implementation cost by 2–16% on average.

To estimate the design time in the conventional approach and the ASMD–FSMD technique, all the designs were created by the same developer. The times (min) spent to each design are given in Table 5.

Analysis of Table 5 shows that the use of the ASMD–FSMD technique can significantly (by a factor of 6.25 on average) decrease the design time. This is due to the fact that, in the ASMD–FSMD technique, there is no need to design (write code and

Table 1. Results of investigations of the implementation of multiplication algorithms

Example	Parameter					
	L_T	L_A	L_T/L_A	t_T	t_A	t_T/t_A
mult_a_4	36	28	1.29	24.46	17.49	1.40
mult_a_8	66	49	1.35	46.96	31.15	1.51
mult_a_16	123	90	1.37	80.08	84.08	0.95
mult_a_32	236	172	1.37	219.03	191.79	1.14
mult_a_64	464	335	1.38	598.42	582.23	1.03
mult_a_128	919	627	**1.47**	4136.21	3969.23	1.04
mult_b_4	36	26	1.38	23.42	16.39	1.43
mult_b_8	64	49	1.30	50.47	47.81	1.06
mult_b_16	121	89	1.36	89.32	106.86	0.84
mult_b_32	234	170	1.38	218.65	178.43	1.23
mult_b_64	461	332	1.39	603.31	580.67	1.04
mult_b_128	913	653	1.40	4093.18	3916.21	1.05
mult_c_4	35	35	1.00	40.25	20.22	1.99
mult_c_8	60	65	0.92	93.11	47.82	1.95
mult_c_16	112	138	0.81	183.55	97.72	1.88
mult_c_32	208	270	0.77	452.46	238.75	1.98
mult_c_64	405	540	0.75	1267.69	639.39	1.98
mult_c_128	793	1006	0.79	8216.11	4200.59	1.96
mult_d_4	32	29	1.10	41.77	20.67	2.02
mult_d_8	58	48	1.21	82.48	36.41	2.27
mult_d_16	98	75	1.31	159.26	53.86	**2.96**
mult_d_32	176	141	1.25	410.87	158.33	2.59
mult_d_64	338	271	1.25	940.51	355.62	2.64
mult_d_128	663	528	1.26	4851.80	2178.69	2.23
booth_4	44	38	1.16	37.11	30.37	1.22
booth_8	82	69	1.19	80.73	52.54	1.54
booth_16	152	125	1.21	111.02	111.21	1.00
booth_32	301	238	1.26	263.55	228.73	1.15
booth_64	590	464	1.27	794.75	654.96	1.21
booth_128	1168	917	1.27	4474.04	4245.59	1.05
mbooth_4	65	68	0.96	25.72	18.25	1.41
mbooth_8	122	138	0.88	36.89	31.59	1.17
mbooth_16	236	278	0.85	68.81	62.48	1.10
mbooth_32	467	492	0.95	150.53	124.27	1.21
mbooth_64	925	973	0.95	413.81	404.31	1.02
mbooth_128	1842	1935	0.95	2396.53	2236.75	1.07

mult_a_N, *mult_b_N*, *mult_c_N*, and *mult_d_N* are the designs implementing multiplication algorithms *a*, *b*, *c*, and *d*, respectively; *booth_N* and *mbooth_N* are the designs implementing the Booth multiplication algorithm and the modified Booth multiplication algorithm; *N* is the width of the multiplier input words (bits); L_T and L_A are the number of FPGA logic elements used (implementation cost) for the conventional approach and the ASMD–FSMD technique, respectively; t_T and t_A are the times of the multiplication operation (nanoseconds) for the conventional approach and the ASMD–FSMD technique, respectively; and L_T/L_A and t_T/t_A are the ratios between the parameters.

Table 2. Results of investigations of the PIC processor implementation

Example	Parameter					
	L_T	L_A	L_T/L_A	F_T	F_A	F_A/F_T
PIC_1_cycle_4	1002	1047	0.96	70.67	75.60	1.07
PIC_1_cycle_8	1653	1712	0.97	66.80	72.45	1.08
PIC_1_cycle_16	2978	3012	0.99	67.85	68.02	1.00
PIC_1_cycle_32	5561	5645	0.99	53.58	66.86	1.25
PIC_1_cycle_64	10804	10925	0.99	52.59	56.16	1.07
PIC_1_cycle_128	21290	21543	0.99	39.02	42.29	1.08
PIC_multi_cycle_4	920	1359	0.68	73.17	76.09	1.04
PIC_multi_cycle_8	1585	2160	0.73	70.64	74.43	1.05
PIC_multi_cycle_16	2913	3502	0.83	62.79	71.41	1.14
PIC_multi_cycle_32	5506	6157	0.89	62.41	69.31	1.11
PIC_multi_cycle_64	10719	11304	0.95	55.22	68.29	1.24
PIC_multi_cycle_128	21204	21849	0.97	41.97	59.41	1.42
PIC_2_cycle_4	1009	1344	0.75	70.53	79.47	1.13
PIC_2_cycle_8	1654	2143	0.77	68.25	73.03	1.07
PIC_2_cycle_16	2980	3497	0.85	58.67	75.34	1.28
PIC_2_cycle_32	5596	6104	0.92	52.72	69.59	1.32
PIC_2_cycle_64	10792	11300	0.96	53.26	68.61	1.29
PIC_2_cycle_128	21287	21834	0.97	41.20	59.96	**1.46**
PIC_4_cycle_4	998	1325	0.75	95.22	70.49	0.74
PIC_4_cycle_8	1670	2198	0.76	90.16	69.52	0.77
PIC_4_cycle_16	2984	3477	0.86	85.42	67.82	0.79
PIC_4_cycle_32	5592	6111	0.92	77.91	67.34	0.86
PIC_4_cycle_64	10829	11260	0.96	62.70	63.71	1.02
PIC_4_cycle_128	21308	21550	0.99	55.66	55.63	1.00

F_T and F_A are the maximum processor synchronization frequency (MHz) for the conventional approach and the ASMD–FSMD technique; F_T/F_A is the ratio between the parameters; and L_T, L_A, and L_T/L_A have the same value.

Table 3. Root-mean-square ratios L_T/L_A and t_T/t_A for different multiplication methods

Method	Value	
	$mid(L_T/L_A)$	$mid(t_T/t_A)$
mult_a	1.37	1.18
mult_b	1.37	1.11
mult_c	0.84	**1.96**
mult_d	1.23	**2.45**
booth	1.23	1.20
mbooth	0.92	1.16
mid	1.16	1.51

mid is the average value of the parameter.

Table 4. Arithmetic means of the ratios L_T/L_A and F_A/F_T for the PIC processor designs

Method	Value	
	$mid(L_T/L_A)$	$mid(F_A/F_T)$
PIC_1_cycle	0.98	1.09
PIC_1_multi	0.84	1.17
PIC_2_cycle	0.87	**1.26**
PIC_4_cycle	0.87	0.86
mid	0.89	1.10

Table 5. Design time

Method	Value		
	DT_T	DT_A	DT_T/DT_A
mult_a	188	33	5.70
mult_b	176	29	6.07
mult_c	172	28	6.14
mult_d	237	43	5.51
booth	167	27	6.19
mbooth	195	35	5.57
PIC_1_cycle	10560	1440	7.33
PIC_1_multi	6240	960	6.50
PIC_2_cycle	840	120	7.00
PIC_4_cycle	980	150	6.53
mid	–	–	6.35

DT_T is the design time in the conventional approach, DT_A is the design time in the ASMD–FSMD technique, DT_T/DT_A is the ratio between the parameters, and *mid* is the average value.

debug) each datapath component, the datapath itself, the control unit, and the main design module.

The experimental investigations allowed us to draw the following conclusions:

(i) In the design of digital devices, the behavior of which is presented well in the form of an operation algorithm, the ASMD–FSMD technique is advantageous in the implementation cost and operation speed over the conventional approach.

(ii) In the design of complex operation units (processors, digital filters, etc.), the conventional approach and special design methods may be ahead of the ASMD–FSMD technique.

(iii) In all the cases, the ASMD–FSMD technique is advantageous over the conventional approach in design time.

CONCLUSIONS

By the example of designing synchronous multipliers and PIC processors, two technologies for designing digital devices were considered: the conventional approach and the ASMD–FSMD technique.

The ASMD–FSMD technique can be used to design digital devices on any element base (not necessarily FPGA), for example, on an ASIC. Any hardware description language (not necessarily Verilog), such as VHDL or SystemVerilog, can be used as a design language.

The ASMD–FSMD technique requires further improvement by:

(i) Using the features of design languages, for example, the use of several blocking operators to the same register in one ASMD unit [22].

(ii) The development of methods for increasing the ASMD operation speed, for example, by decreasing several states in the algorithm cycles.

(iii) The development of methods for optimizing the ASMD to improve the efficiency and reliability and reduce the implementation cost.

In addition, it seems promising to use the ASMD–FSMD technique for the high-level complex design.

FUNDING

This study was supported in part by the Bialystok University of Technology (Bialystok, Poland), project no. WZ/WI-IIT/4/2020.

REFERENCES

1. D. D. Gajski, N. D. Dutt, A. C. Wu, and S. Y. Lin, *High-Level Synthesis: Introduction to Chip and System Design* (Kluwer, Boston, 1992).
2. R. Auletta, B. Reese, and C. Traver, in *Proc. Int. Conf. on Computer Design ICCD'93, Cambridge (MA), Oct. 3–6, 1993* (IEEE, New York, 1993), p. 178.
3. C. Karfa, D. Sarkar, and C. Mandal, IEEE Trans. Comput.-Aided Des. **29**, 479 (2010).
4. J. Hu, G. Wang, G. Chen, and X. Wei, IEEE Access. **7**, 183435 (2019).
5. P. Schaumont, S. Shukla, and I. Verbauwhede, in *Proc. Design Automation & Test in Europe Conf. Verona., July 11–14, 2005* (IEEE, New York, 2006), Vol. 1, p. 6.
6. J. Zhu and D. D. Gajski, in *Proc. 7th Int. Workshop on Hardware/Software Codesign CODES'99, Rome, Mar. 3, 1999* (IEEE, New York, 1999), p. 121.
7. N. Kavvadias and K. Masselos, in *Proc. Int. Conf. on Application-Specific Systems, Architectures and Processors, Delft, July 9–11, 2012* (IEEE, New York, 2012), p. 157.
8. K. Banerjee, D. Sarkar, and C. Mandal, IEEE Trans. Comput.-Aided Des. **33**, 2015 (2014).
9. E. Hwang, F. Vahid, and Y. C. Hsu, in *Proc. Int. Conf. on Design, Automation and Test in Europe, Munich, Mar. 9–12, 1999* (Int. Conf., Munich, 1999), p. 7.
10. A. Ch. Abdullah, Ch. Y. Ooi, N. Bt. Ismail, and N. B. Mohammadd, in *Proc. Int. Symp. on Circuits and Systems (ISCAS), Montreal, May 22–25, 2016* (IEEE, New York, 2016), p. 1942.
11. R. Babakov, A. Barkalov, and L. Titarenko, in *Proc. Int. Conf. on Experience of Designing and Application of CAD Systems in Microelectronics (CADSM), Lviv, Feb. 21–25, 2017* (IEEE, New York, 2017), p. 203.
12. C. R. Clare, *Designing Logic Systems Using State Machines* (McGraw-Hill Book Company, New York, 1973).

13. D. H. Green and M. A. Chughtai, IEE Proc. E-Comput. & Digital Tech. **133**, 194 (1986).
14. S. Baranov, in *Proc. Int. Conf. EUROMICRO, Vasteras, Aug. 27–27, 1998* (IEEE, New York, 1998), Vol. 1, p. 176.
15. M. Jenihhin, S. Baranov, J. Raik, and V. Tihhomirov, in *Proc. Int. Conf. Latin American Test Workshop (LATW), Quito, Apr. 10–13, 2012* (IEEE, New York, 2012), p. 1.
16. M. D. Ciletti, *Advanced digital design with the Verilog HDL* (Prentice Hall of India, New Delhi, 2005).
17. P. Martin, E. Bueno, F. J. Rodriguez, and V. Sáez, in *Proc. Annual Conf. IEEE Industrial Electronics. Porto, Nov. 3–5, 2009* (IEEE, New York, 2009), p. 2811.
18. A. Saha, A. Ghosh, and K. G. Kumar, in *Proc. Int. Conf. on Advances in Science and Technology, Bangkok, Jan. 19–22, 2017* (Elsevier, Bangkok, 2017), p. 138.
19. P. Burciu, J. Electrical Engineering, Electronics, Control & Comput. Sci. **5** (3), 1 (2019).
20. K. B. Sowmya, G. Shreyans, and R. T. Vishnusai, in *Proc. Int. Conf. on Communication and Electronics Systems. Coimbatore, June 10–12, 2020* (IEEE, New York, 2020), p. 176.
21. V. V. Solov'ev, *Design of Functional Blocks of the Built-in Systems on FPGA* (Goryachaya Liniya–Telekom, Moscow, 2020).
22. V. V. Solov'ev, *The Verilog Language in Design of the Built-in Systems on FPGA* (Goryachaya Liniya–Telekom, Moscow, 2020).

Translated by E. Bondareva